

HADES

Command-Line Flags — Cheat Sheet
Run with: `python hades.py [flags]` · 2026-06-16

All available flags

Flag	Short	Value	What it does
<code>--url</code>	<code>-u</code>	URL	Target URL to scan. Must start with <code>http://</code> or <code>https://</code> . If omitted, Hades asks for it interactively.
<code>--profile</code>	<code>-p</code>	PROFILE	Which set of modules to run: quick, passive, cms, full, db_scan, ai_scan, engage, oob_scan, tls_scan. If omitted, an interactive menu is shown (menu option 8 = CVE Vulnerability Intelligence).
<code>--module</code>	<code>-m</code>	NAME	Run ONE module only (e.g. headers_check). Overrides <code>--profile</code> . Great for quick targeted checks.
<code>--no-open</code>	<code>-</code>	(flag)	Do not auto-open the HTML report in a browser when the scan finishes (the HTML + JSON reports are still written).
<code>--arsenal</code>	<code>-</code>	(flag)	Open the RedTeam Arsenal — a searchable page of offensive tools by attack type. No scan (also menu option 666).
<code>--skills</code>	<code>-</code>	(flag)	Open the Skills Library — a searchable page of the expert playbooks Hades draws on, by subdomain. No scan (also menu option 10).
<code>--exploit</code>	<code>-</code>	(flag)	After the scan, launch sqlmap against any CONFIRMED SQL injection. Requires sqlmap. Authorised targets only.
<code>--bruteforce</code>	<code>-</code>	(flag)	Opt-in: spray common credentials against discovered login forms and HTTP Basic-Auth. Authorised targets only.
<code>--oob-host</code>	<code>-</code>	HOST	Reachable callback address for oob_scan (public IP / tunnel). Auto-detected if omitted.
<code>--oob-port</code>	<code>-</code>	PORT	Port for the out-of-band callback listener (oob_scan). 0 = auto-pick a free port.
<code>--proxy</code>	<code>-</code>	URL	Route all traffic through an HTTP/HTTPS proxy, e.g. Burp Suite at <code>http://127.0.0.1:8080</code> .
<code>--threads</code>	<code>-t</code>	N	Number of concurrent worker threads (default: 10). Higher = faster but noisier/heavier.
<code>--ignore-robots</code>	<code>-</code>	(flag)	Ignore robots.txt Disallow rules during active scanning. Use only on targets you fully control.
<code>--wordlist</code>	<code>-w</code>	FILE	Use a custom wordlist file instead of the built-in lists (affects dir_scan / admin_panel).
<code>--cookies</code>	<code>-</code>	STRING	Send a Cookie header, e.g. <code>"session=abc123; token=xyz"</code> . Use to scan as a logged-in user.
<code>--auth-token</code>	<code>-</code>	TOKEN	Send an Authorization: Bearer <TOKEN> header. Use for API / token-protected targets.

Flag	Short	Value	What it does
--login-url	—	URL	Log in before scanning so the crawler + active modules run authenticated (form action / login page). Use with --login-data.
--login-data	—	STRING	Login credentials as form data, e.g. "username=admin&password=secret". CSRF hidden fields are auto-replayed.
--login-check	—	STRING	Text that proves the session is authenticated (e.g. "Logout") — confirms the login worked.
--help	-h	(flag)	Show the built-in help message listing all flags, then exit.

Scan profiles (--profile)

Profile	What it runs
quick	Fast surface scan: basic info, security headers, SSL/TLS, robots.txt.
passive	All reconnaissance + passive web checks. No active probing (no SQLi/XSS/brute-force).
cms	CMS-focused: CMS detection, admin panels, and CVE mapping.
full	Everything — the most thorough scan (this is the default behaviour).
tools	External-tool integrations suite (also interactive menu option 12): runs Nmap (service/version), Gobuster (content discovery), theHarvester and Recon-ng (OSINT) when installed, and writes a Maltego-importable CSV of the discovered entities. Each tool degrades to an install hint if absent.
db_scan	Red-team database security audit: DB ports, unauth access + data extraction, SQL/NoSQL injection, leaked secrets/connection strings, GraphQL, admin GUIs, dumps; emits a DB exposure score, attack path and loot. Pair with --exploit to launch sqlmap on confirmed SQLi.
ai_scan	Red-team AI/LLM audit: SDK/provider fingerprint, exposed AI keys, unauthenticated local LLM servers, exposed AI UIs and the prompt-injection surface (OWASP LLM Top 10 + MITRE ATLAS).
engage	Active exploitation engagement: confirms bugs then proves impact with benign payloads (RCE via id/uname, LFI read, SSRF to cloud metadata), writing evidence to loot/. Authorisation-gated.
oob_scan	Out-of-band (OAST) blind-vuln detection: blind SSRF / RCE / stored XSS via a self-hosted callback listener; auto public tunnel (cloudflared/ngrok) so it works behind NAT.
cve_scan	CVE Vulnerability Intelligence (also interactive menu option 8): fingerprints the stack, matches real CVEs (2020+) from a local KEV/EPSS/NVD database, ranked by a Hades CVE Priority Score. Build the full offline corpus once with tools/build_vulndb.py.
tls_scan	Offensive TLS/SSL audit (also interactive menu option 9) via SSLyze: legacy protocols, weak/anon ciphers, no forward secrecy, certificate trust/expiry/hostname issues, TLS compression, insecure renegotiation, and Heartbleed/ROBOT/CCS injection. Handshake-only; needs the optional sslyze package.

Module names (--module NAME)

Use any of these with **--module** to run just that one check:

Recon: `basic_info` · `whois_lookup` · `dns_check` · `ssl_check` · `port_scan` · `waf_detect` · `tech_stack` ·
`js_recon` · `cloud_buckets` · `git_dumper` · `wayback`
Web: `headers_check` · `robots_txt` · `sitemap` · `cms_detect` · `admin_panel` · `dir_scan` · `subdomain_scan` ·
`broken_links` · `http_methods` · `backup_files` · `sensitive_files` · `cookie_analysis` · `redirect_chain` ·
`email_exposure` · `favicon_hash` · `cors_check` · `clickjacking` · `dir_listing` · `blacklist_check` · `screenshot`
Vulns: `sqli_detect` · `xss_detect` · `command_injection` · `ssti_detect` · `lfi_detect` · `open_redirect` ·
`ssrf_detect` · `jwt_attacks` · `auth_bypass` · `idor_detect` · `bruteforce` · `cve_mapping` · `default_creds`
DB: `db_security` (run via `--profile db_scan` – dedicated red-team database audit)
CVE: `cve_vulnerability` (menu option 8 – CVE intelligence; build the corpus with `tools/build_vulndb.py`)
TLS: `hephaestus_tls` (menu option 9 / `--profile tls_scan` – offensive TLS audit via SSLyze)

Example commands

Interactive menu (asks URL + scan type)

```
python hades.py
```

Quick scan of a site

```
python hades.py --url https://example.com --profile quick
```

Full scan (HTML + JSON written automatically)

```
python hades.py -u https://example.com -p full
```

Run only one module

```
python hades.py -u https://example.com -m headers_check
```

Scan through Burp Suite proxy

```
python hades.py -u https://example.com --proxy http://127.0.0.1:8080
```

Authenticated scan (form login + CSRF)

```
python hades.py -u https://example.com --login-url https://example.com/login --login-data "username=admin&password=secret" --login-check "Logout"
```

Authenticated scan (with a cookie)

```
python hades.py -u https://example.com --cookies "session=abc123"
```

More threads (faster)

```
python hades.py -u https://example.com -t 25
```

Detect + launch sqlmap on SQLi

```
python hades.py -u "http://testaspnet.vulnweb.com/Comments.aspx?id=1" --exploit
```

Database security audit

```
python hades.py -u https://example.com -p db_scan
```

DB audit + auto-exploit confirmed SQLi

```
python hades.py -u http://testaspnet.vulnweb.com -p db_scan --exploit
```

CVE intelligence (interactive menu, option 8)

```
python hades.py -u https://example.com
```

Build the full offline CVE corpus (once)

```
python tools/build_vulndb.py
```

Offensive TLS/SSL audit (SSLyze)

```
python hades.py -u https://example.com -p tls_scan
```

RedTeam Arsenal (no scan)

```
python hades.py --arsenal
```

Skills Library (no scan)

```
python hades.py --skills
```

Show all flags

```
python hades.py --help
```